

ProductionManagementAI — Completion Tracking and Dashboard Queries — Database Design Document (テーブル定義書)

003_DB — requirements REQ-028–REQ-039 (`work-items/WI-004/brief.md`), basic design 003_BD (`docs/en/010_basic-design/003/003_BD_production-dashboard.md`), implements 003_DD and 003_DD-API (`docs/en/020_detailed-design/` , to be written) and the REQ-033 amendment to 001_DD.

This document changes one table 001_DB defined: it adds **one column and two check constraints** to `production_orders` , two partial indexes, and demo history. Everything 001_DB and 002_DB state about `products` , `production_orders` and `production_order_number_counters` stays in force and is not restated.

Physical naming follows 000_DB–002_DB: `snake_case` , `timestamptz` UTC timestamps suffixed `_utc` , EF Core `pk_` / `fk_` / `ix_` / `ck_` constraint names.

Document control (改版履歴)

Version	Date	Author	Revision content
1	2026-09-22	Claude (for ThanhTN)	Initial creation (WI-004 DEC-003, DEC-004, DEC-013–DEC-015)

Table list

Table	Physical name	Purpose	Change in 003_DB
Products	<code>products</code>	Product reference data (001_DB)	none — read only, joined for product labels
Production orders	<code>production_orders</code>	One row per production order (001_DB)	new column <code>completed_at_utc</code> ; two check constraints; two partial indexes; backfill; demo history
Production order number counters	<code>production_order_number_counters</code>	Order-number sequence per year (001_DB)	the seed year's counter advances past the new demo orders

ER diagram and relationships

Unchanged from 001_DB. The dashboard traverses one relationship:

Table	Related table	Relationship (1:1 / 1:N / N:N)	FK column
products	production_orders	1:N	production_orders.product_id

Table definitions

production_orders — added column

Item name	Physical name	Data type	Length	NOT NULL	Default	PK/FK	Notes
CompletedAtUtc	completed_at_utc	timestamptz	—	no	none	—	When the order became <code>Completed</code> (REQ-033, DEC-003). Set by the application in the same save that moves the order from <code>InProgress</code> to <code>Completed</code> , to that save's UTC time — the same value it writes to <code>updated_at_utc</code> . Never changed afterwards, never accepted from a request. <code>NULL</code> for every other status. Existing completed orders are backfilled (see Migration impact)

Why a column and not a history table: `Completed` is reachable only from `InProgress` and is terminal (001_BD M-02), so an order is completed at most once and one timestamp holds the whole fact. DEC-003 records the trade-off.

Columns the dashboard reads

Physical name	Used for
status	Status tiles (REQ-029); "active" = <code>Draft</code> or <code>InProgress</code>
due_date	Overdue / due-soon groups (D-01, D-02), workload buckets (D-03), on-time comparison (D-07)
quantity	Workload and top-products quantities (D-03, D-04), completed quantities (D-05, D-06)
product_id	Top-products grouping (D-04); join to <code>products</code> for labels
created_at_utc	Lead time (D-09)
completed_at_utc	Every delivery metric (D-05–D-09)
id, order_number	The at most 20 listed orders (D-01, D-02); tie-breaker

`notes`, `updated_at_utc`, `order_year`, `order_seq` and `xmin` are not read.

Constraints

Constraint	Type (PK/FK/UNIQUE/CHECK)	Table.column(s)	References	Rule
<code>ck_production_orders_completed_at_matches_status</code>	CHECK	<code>production_orders.status</code> , <code>completed_at_utc</code>	—	<code>(status = 'Completed') =</code> <code>(completed_at_utc IS NOT NULL)</code> — a completed order always has a completion time and no other order has one (REQ-033 failure criterion)
<code>ck_production_orders_completed_at_not_before_created</code>	CHECK	<code>production_orders.completed_at_utc</code> , <code>created_at_utc</code>	—	<code>completed_at_utc IS NULL OR</code> <code>completed_at_utc >= created_at_utc</code> — no lead time can be negative (REQ-037 failure criterion), whatever wrote the row

Unlike the rules 001_DB leaves to the application (due date $\geq$ today, the transition graph), both of these hold at every moment regardless of the clock, so the database can enforce them. They are a safety net under the application rule, not a replacement for it: the entity sets the value (001_DD module 1, amended), and a code path that forgot to would fail its save loudly instead of silently producing an order missing from the delivery figures.

The transition rule itself (`InProgress` $\rightarrow$ `Completed` only; `Completed` terminal) stays application-enforced as 001_DB decided. Because `Completed` is terminal, the first constraint also means `completed_at_utc` can never be cleared by a legal transition.

Access patterns

The dashboard issues **seven statements per load** (Q1–Q6, with Q3 run once per attention group), all inside one read-only `REPEATABLE READ` transaction so they see one snapshot (DEC-015; see Transactions and concurrency). Every value that depends on the date is computed by the application from the plant clock and passed as a parameter; the plant timezone name itself is passed as a parameter (`@zone` , from `PlantOptions.TimeZone`) where the SQL must convert a timestamp to a plant-local date. No input from the client reaches any statement — the endpoint has no parameters (003_BD §5).

Parameters, all computed once per snapshot from `IPlantClock` (T = plant today, W = Monday of T's week):

Parameter	Value	Used by
<code>@today</code>	T (date)	Q2–Q4
<code>@soonEnd</code>	T + 7 (date)	Q3 (due soon)

Parameter	Value	Used by
@week0	W(T) (date)	Q2 (workload buckets)
@trendStartUtc	start of plant-local day W(T) – 77, as a UTC instant	Q5, Q6 lower bound
@weekStartUtc	start of plant-local day W(T), UTC	Q5 (completed this week)
@monthStartUtc	start of plant-local day 1 of T's month, UTC	Q5 (completed this month)
@windowStartUtc	start of plant-local day T – 29, UTC	Q5 (on time, lead time)
@endUtc	start of plant-local day T + 1, UTC	Q5, Q6 upper bound
@zone	Asia/Tokyo (configured)	Q5, Q6 plant-date conversion

@trendStartUtc is always the earliest lower bound: the trend reaches back at least 77 days, while the month and the 30-day window reach back at most 30. Range predicates are written on completed_at_utc itself, against UTC instants, so the index below serves them; the plant-date conversion appears only in the grouping and in the on-time comparison, where it runs on the already-narrowed rows.

Q1 — status counts (REQ-029)

```
SELECT status, count(*) FROM production_orders GROUP BY status;
```

Q2 — workload buckets (D-03, REQ-031), active orders only

```
SELECT CASE
    WHEN due_date < @today          THEN -1          -- Overdue
    WHEN due_date >= @week0 + 56    THEN  8          -- Later
    ELSE (due_date - @week0) / 7    -- week 0..7
END AS bucket,
count(*), sum(quantity)
FROM production_orders
WHERE status IN ('Draft', 'InProgress')
GROUP BY bucket;
```

`due_date - @week0` is an integer number of days, so the integer division places every date from T to W(T) + 55 in exactly one of weeks 0–7. The application fills buckets with no row as zero (REQ-031). The overdue bucket's count is also D-01's total, which Q3 reads independently — both come from the same snapshot, so they agree.

Q3 — the attention groups (D-01, D-02, REQ-030), run twice with different bounds

```
SELECT o.id, o.order_number, o.quantity, o.due_date, o.status, p.sku, p.name,
       count(*) OVER () AS total
FROM production_orders o
JOIN products p ON p.id = o.product_id
WHERE o.status IN ('Draft', 'InProgress')
      AND o.due_date < @today                -- overdue
      -- or: AND o.due_date BETWEEN @today AND @soonEnd    -- due soon
ORDER BY o.due_date, o.order_number
LIMIT 10;
```

`count(*) OVER ()` is evaluated before `LIMIT`, so one statement returns both the first 10 rows and the group's total. When the group is empty it returns no row and the total is 0. (Listed as Q3a and Q3b below.)

Q4 — top products (D-04, REQ-032)

```
SELECT p.id, p.sku, p.name, sum(o.quantity) AS open_quantity, count(*) AS active_orders
FROM production_orders o
JOIN products p ON p.id = o.product_id
WHERE o.status IN ('Draft', 'InProgress')
GROUP BY p.id, p.sku, p.name
ORDER BY open_quantity DESC, p.sku
LIMIT 10;
```

Q5 — delivery figures (D-05, D-06, D-07, D-09; REQ-034, REQ-035, REQ-037), one row

```

SELECT
  count(*)      FILTER (WHERE completed_at_utc >= @weekStartUtc) AS week_count,
  sum(quantity) FILTER (WHERE completed_at_utc >= @weekStartUtc) AS week_quantity,
  count(*)      FILTER (WHERE completed_at_utc >= @monthStartUtc) AS month_count,
  sum(quantity) FILTER (WHERE completed_at_utc >= @monthStartUtc) AS month_quantity,
  count(*)      FILTER (WHERE completed_at_utc >= @windowStartUtc) AS window_count,      -- y
  count(*)      FILTER (WHERE completed_at_utc >= @windowStartUtc
                        AND (completed_at_utc AT TIME ZONE @zone)::date <= due_date)
                        AS window_on_time,      -- x
  avg(extract(epoch FROM completed_at_utc - created_at_utc) / 86400.0)
      FILTER (WHERE completed_at_utc >= @windowStartUtc) AS window_lead_days
FROM production_orders
WHERE completed_at_utc >= @trendStartUtc
  AND completed_at_utc < @endUtc;

```

`sum` and `avg` over no rows return `NULL`; the application maps `NULL` sums to 0 and a `NULL` average (with `window_count = 0`) to "—" (003_BD M-13, M-14). Rounding and presentation are the application's, so the database returns the unrounded mean.

Q6 — completion trend (D-08, REQ-036)

```

SELECT date_trunc('week', completed_at_utc AT TIME ZONE @zone)::date AS week_start, count(*)
FROM production_orders
WHERE completed_at_utc >= @trendStartUtc
  AND completed_at_utc < @endUtc
GROUP BY week_start;

```

PostgreSQL's `date_trunc('week', ...)` truncates to the ISO week's Monday, which is W(d) exactly (DEC-008). The application lays the result onto the 12 expected Mondays W(T) – 77, ..., W(T), filling missing weeks with zero.

`@endUtc` bounds Q5 and Q6 so that a completion timestamp later than the snapshot's today cannot enter a figure. That cannot happen in production. It matters in tests, which pin the plant clock independently of the database's `now()`.

Access pattern	Frequency	Served by
Q1 status counts	once per dashboard load	sequential scan of <code>production_orders</code> — see Performance
Q2, Q4 active-order aggregates	once each per load	<code>ix_production_orders_active_due_date</code> (bitmap scan of the active set)
Q3a overdue, Q3b due soon: first 10 by due date	once each per load	<code>ix_production_orders_active_due_date</code> in index order
Q5, Q6 recent completions	once each per load	<code>ix_production_orders_completed_at_utc</code> range scan

Index definitions

Existing indexes from 001_DB and 002_DB are unchanged. New in 003_DB:

Index name	Table	Column(s)	Type	Rationale (access pattern)
<code>ix_production_orders_active_due_date</code>	<code>production_orders</code>	<code>due_date,</code> <code>order_number</code> WHERE <code>status IN ('Draft',</code> <code>'InProgress')</code>	btree, partial	Q2–Q4 read only active orders. Active orders are the open workload, which stays roughly constant while <code>Completed</code> and <code>Cancelled</code> orders accumulate forever, so over time the active set becomes a small fraction of the table. The partial index keeps these queries proportional to the open workload rather than the table's history. Its <code>(due_date,</code> <code>order_number)</code> order is exactly Q3's <code>ORDER BY</code> , so each attention group is read in order and stops after 10 rows. 002_DB's full <code>ix_production_orders_due_date_order_number</code> would serve Q3 too, but it walks every completed and cancelled order with an old due date before reaching the first overdue active one
<code>ix_production_orders_completed_at_utc</code>	<code>production_orders</code>	<code>completed_at_utc</code> WHERE <code>completed_at_utc IS</code> <code>NOT NULL</code>	btree, partial	Q5 and Q6 read completions from the last ~12 weeks, out of every completion ever recorded. A range scan on this index reads only that window. The partial predicate leaves out every non-completed row, which would otherwise all be <code>NULL</code> entries

Indexes deliberately not added

Candidate	Why not
<code>status</code> (for Q1)	Q1 needs every row's status, so an index could only help as an index-only scan, and the visibility map on a table that is updated all day makes that unreliable. At the sizes in Performance, a scan of the table is cheap. Revisit past roughly a million rows, where a maintained per-status counter would be the better answer, not an index
<code>(product_id) WHERE active</code> (for Q4)	Q4 aggregates the whole active set whichever index finds it; grouping by product is a hash aggregate over that set. The active partial index already bounds the rows read

Candidate	Why not
(completed_at_utc) INCLUDE (quantity, due_date, created_at_utc) covering index	Would make Q5/Q6 index-only, but it widens an index on a table updated all day, to save heap fetches on at most ~12 weeks of completions. Not worth it at this volume

Transactions and concurrency

Operation	Transaction scope	Concurrency control
Dashboard snapshot (Q1–Q6, FN-017)	One explicit transaction, BEGIN ISOLATION LEVEL REPEATABLE READ READ ONLY ... COMMIT (DEC-015)	In PostgreSQL, REPEATABLE READ gives every statement in the transaction the same snapshot, taken at the first statement. So all seven statements see one consistent state of production_orders , and 003_BD's cross-checks hold: the tiles sum to the total, and the workload bars sum to Draft + In progress. A read-only transaction never raises a serialization failure at this level, so there is no retry path. It takes no locks beyond ACCESS SHARE and blocks no writer
Complete an order (SCR-001 save, FN-021)	The existing single SaveChangesAsync of 001_DD-FN UpdateAsync	Unchanged: the xmin optimistic-concurrency check (WI-002 DEC-010, DEC-014). completed_at_utc is one more column in the same UPDATE , so the completion time and the status change commit together or not at all

Why not one statement with CTEs instead of a transaction: it would give the same consistency, but it merges seven differently shaped results (a grouped count, two row lists, a ranking, a single row, a weekly series) into one result set that EF Core cannot map without contortions. A read-only snapshot transaction keeps each query readable and testable on its own.

Demo seed data

DEC-004 (user) requires the seeded demo orders to get plausible completion times so the delivery widgets have data. With WI-003's seed alone that is not possible. Only 16 of its 80 orders are Completed , and every seeded order was created at most 49 days before the seed ran; a completion cannot precede creation. So those 16 could fill at most the last 7 of the trend's 12 weeks, at about two per week. DEC-013 (user) therefore does **both**: it re-dates the 16 existing completed orders and adds 40 historical completed orders. DEC-014 adds 4 active orders due far enough ahead to populate the last two week bars and the "Later" bar, which WI-003's seed (due dates at most 40 days out) does not reliably reach.

As in 002_DB (WI-003 DEC-011), every date is **relative to the moment the migration runs**. Everything else is a constant: ids, products, quantities and the day offsets.

Re-dated existing completed orders (16)

Seeded rows `seq` 6, 12, 18, 24, 30, 31, 37, 43, 49, 55, 61, 67, 68, 73, 74, 80, identified by WI-003's seed id prefix and status `Completed`. Their product, quantity and **due date are not changed**, so Screen B's due-date filters and overdue markers are unaffected. The migration sets `completed_at_utc = now() - c days`, `updated_at_utc = completed_at_utc` (the completion save is the order's last update), and `created_at_utc = completed_at_utc - 1 days`.

seq	Due (days from run date, unchanged)	Completed <i>c</i> days ago	Lead <i>l</i> days	Created days ago	On time?
74	+0	0	9	9	on time
43	+5	1	12	13	on time
24	-6	2	15	17	late
49	-7	3	11	14	late
80	-10	4	18	22	late
18	+10	6	8	14	on time
37	+11	8	14	22	on time
12	+16	10	21	31	on time
73	+15	12	10	22	on time
31	+17	14	16	30	on time
67	+21	17	13	30	on time
6	+22	20	19	39	on time
61	+27	23	9	32	on time
55	+33	26	22	48	on time
30	+38	29	12	41	on time
68	+40	33	17	50	on time

An order finished well before its due date is plausible — and it is on time (003_BD D-07).

New historical completed orders (40)

All `Completed`, no notes. For each: `completed_at_utc = now() - c days`, `created_at_utc = completed_at_utc - 1 days`, `updated_at_utc = completed_at_utc`, and `due_date` = the completion's plant-local date + δ days (negative δ = completed late).

#	Product	Qty	Completed c days ago	Lead l days	Due δ vs completion	On time?
1	P-1001	320	0	11	+3	on time
2	P-1002	540	2	14	-2	late
3	P-1003	150	3	9	+4	on time
4	P-1004	880	5	16	+1	on time
5	P-1005	95	7	12	+0	on time
6	P-1006	1,200	9	20	-3	late
7	P-1007	430	11	8	+5	on time
8	P-1008	260	13	15	+2	on time
9	P-1009	710	15	10	-1	late
10	P-1010	55	16	18	+6	on time
11	P-1011	1,900	19	13	+3	on time
12	P-1012	340	21	9	+1	on time
13	P-1013	620	24	22	-4	late
14	P-1014	180	25	11	+2	on time
15	P-1015	2,500	27	14	+7	on time
16	P-1016	75	30	17	+0	on time
17	P-1017	960	32	12	-2	late
18	P-1018	410	35	19	+4	on time
19	P-1019	130	37	10	+1	on time
20	P-1020	1,450	40	24	-5	late
21	P-1021	290	42	13	+3	on time
22	P-1022	820	45	16	+2	on time
23	P-1023	65	47	11	-1	late
24	P-1024	1,100	50	20	+5	on time
25	P-1025	370	53	14	+1	on time

#	Product	Qty	Completed c days ago	Lead l days	Due δ vs completion	On time?
26	P-1026	2,200	56	9	-3	late
27	P-1027	145	58	18	+2	on time
28	P-1028	690	61	12	+4	on time
29	P-1029	310	64	21	-2	late
30	P-1030	1,750	66	15	+3	on time
31	P-1001	85	69	10	+1	on time
32	P-1002	560	71	17	-6	late
33	P-1003	240	74	13	+2	on time
34	P-1004	1,300	76	22	+5	on time
35	P-1005	420	78	11	+0	on time
36	P-1006	95	80	16	-3	late
37	P-1007	780	82	14	+2	on time
38	P-1008	210	83	19	+4	on time
39	P-1009	1,600	85	12	-1	late
40	P-1010	330	88	20	+3	on time

New far-due active orders (4, DEC-014)

#	Product	Qty	Status	Due (days from run date)	Created days ago	Bar on the run date
41	P-1011	600	Draft	+49	5	week 7 (for any run weekday)
42	P-1017	1,800	Draft	+56	8	Later
43	P-1024	450	InProgress	+63	12	Later
44	P-1003	950	Draft	+42	3	week 6 (for any run weekday)

`updated_at_utc = created_at_utc` , `completed_at_utc` `NULL` . Week k spans $W(T) + 7k \dots W(T) + 7k + 6$, which is $T + 7k - wd \dots T + 7k + 6 - wd$ for a run weekday $wd = 0$ (Mon) ... 6 (Sun). So $T + 42$ always lands in week 6 and $T + 49$ in week 7, and $T + 56$ and later are always "Later". Order #44 is needed because WI-003's seed leaves week 6 empty on a Monday or Tuesday run.

What the seed produces

Checked by simulating the rows above for a run on each weekday, and on the 1st, 15th and 28th of a month, with the dashboard viewed on the run day:

Figure	Result
Completion trend (D-08)	Every one of the 12 weeks has at least 2 completions, for every run weekday; rising from about 3 a week to 7–10 in the most recent weeks
On time, last 30 days (D-07)	23 of 30 — 77%, for every run weekday
Average lead time, last 30 days (D-09)	13.7 days, based on 30 orders
Completed this week (D-05)	2 on a Monday run, rising to 10 on a Sunday run — never empty
Completed this month (D-06)	2 on the 1st of a month, up to 29 late in a month
Status tiles	124 orders: 35 Draft , 25 InProgress , 56 Completed , 8 Cancelled
Workload (D-03)	Overdue 15; every week bar 0–7 and Later non-zero on the run day, for every run weekday; bars sum to 60 (35 + 25)
Overdue / due soon (D-01, D-02)	15 overdue (10 listed + "Showing 10 of 15"), 8 due soon on the run day
Top products (D-04)	27 products have open quantity, so the list of 10 is full

The figures drift as days pass after the seed runs, just as WI-003's overdue markers do. Rebuilding the Compose volume re-runs the seed against the new date.

Guards, order numbers and scope

- **Re-dating** touches only rows with WI-003's seed id prefix whose status is still `Completed` . On a database where WI-003's seed never ran, there are none and the step does nothing.
- **Inserting** runs only when WI-003's seeded rows exist, which marks the database as a demo database, and none of the 44 new ids exist yet. Re-applying it is a no-op, and a database of real orders is never given demo history.
- **Ids** are fixed UUIDv7-shaped constants with their own prefix, `0197e4a0-0000-7000-8002-...` , distinct from WI-003's `...-8001-...` , so each seed can be removed on its own.
- **Order numbers** are not fixed. A user may already have created orders after WI-003's seed (taking `00081` onwards), so the new rows take the next 44 sequence numbers after the seed year's counter. The counter is then advanced to the last one, in the same migration, so Screen A continues after them. WI-003's integration test that asserts the first API-created order is `00081` changes to `00125` .
- **Order year**: all new rows use the plant year of the run date, as WI-003's seed does, even though some were created up to 108 days earlier. If the seed runs in January–April, some demo orders therefore carry a year later than their creation date. That is cosmetic, and it is accepted rather than giving the demo orders two counters.

- **Environments:** like WI-003's seed, this reaches every database where WI-003's seed ran, including the integration-test database. Dashboard integration tests must assert against orders they create under a pinned plant clock set far from the seeded dates, or assert differences, never the seeded totals. 003_DD specifies how.

Because these values are computed at run time, the seed is raw SQL (`migrationBuilder.Sql`), for the same reason 002_DB gives.

Application database privileges

No change. The dashboard needs `SELECT` on `production_orders` and `products` , and Screen A's save needs `UPDATE` on the new column. `pmai_app` already has table-level `SELECT, INSERT, UPDATE` on `production_orders` (001_DB), and a table-level grant covers a column added later. No `DELETE` and no DDL is needed or granted. All three migrations below run as the owner.

API / DD mapping

DD field names are 003_BD \$3 variable names; API field names are confirmed in 003_DD-API.

Table.column / query	DD field	API field
Q1 counts by <code>status</code>	<code>statusCounts.*</code> (items 7–11)	<code>statusCounts.total</code> , <code>.draft</code> , <code>.InProgress</code> , <code>.completed</code> , <code>.cancelled</code>
Q3a rows / <code>total</code>	<code>overdue</code> (items 16, 17)	<code>overdue.total</code> , <code>overdue.orders[]</code>
Q3b rows / <code>total</code>	<code>dueSoon</code> (items 18, 19)	<code>dueSoon.total</code> , <code>dueSoon.orders[]</code>
<code>production_orders.id</code> , <code>order_number</code> , <code>quantity</code> , <code>due_date</code> , <code>status</code> ; <code>products.sku</code> , <code>name</code>	listed order fields (M-11, M-12, M-15)	<code>orders[].id</code> , <code>.orderNumber</code> , <code>.quantity</code> , <code>.dueDate</code> , <code>.status</code> , <code>.product.sku</code> , <code>.product.name</code>
Q2 buckets	<code>workload</code> (item 20)	<code>workload[]</code> : { <code>kind</code> : <code>overdue</code> \ <code>week</code> \ <code>later</code> , <code>weekStart?</code> , <code>orderCount</code> , <code>quantity</code> } , 10 entries
Q4 rows	<code>topProducts</code> (item 22)	<code>topProducts[]</code> : { <code>product.sku</code> , <code>product.name</code> , <code>openQuantity</code> , <code>activeOrderCount</code> }
Q5 week / month	<code>completedThisWeek</code> , <code>completedThisMonth</code> (items 12, 13)	<code>completedThisWeek</code> .{ <code>orderCount</code> , <code>quantity</code> } , <code>completedThisMonth</code> .{...}
Q5 <code>window_on_time</code> , <code>window_count</code>	<code>onTime</code> (item 14)	<code>onTime.onTimeCount</code> , <code>onTime.completedCount</code> (the rate is derived for display, not sent twice)

Table.column / query	DD field	API field
Q5 <code>window_lead_days</code> , <code>window_count</code>	<code>leadTime</code> (item 15)	<code>leadTime.averageDays</code> (<code>null</code> when none), <code>leadTime.orderCount</code>
Q6 rows	<code>completionTrend</code> (item 23)	<code>completionTrend[]</code> : { <code>weekStart</code> , <code>orderCount</code> } , 12 entries
— (snapshot time, plant today)	<code>asOf</code> (item 4)	<code>asOf</code> (ISO 8601 UTC), <code>today</code> (plant-local date)
<code>production_orders.completed_at_utc</code>	—	not exposed by any endpoint: Screen A does not show it (REQ-033), and the dashboard exposes only aggregates of it

Performance expectations

Same sizing assumption as 002_DB: about 15,000 orders a year. The active set (the open workload) stays in the hundreds to low thousands; completions in any 12-week window number about 3,500 at most. The demo database holds 124.

Query	Expectation
Q1 status counts	Sequential scan of the whole table plus a 4-group hash aggregate — about a millisecond per 100,000 rows, acceptable for years at this volume
Q2, Q4	Bitmap scan of the active partial index, aggregate over the open workload
Q3a, Q3b	Ordered scan of the active partial index, stopping after 10 rows; the window count needs the whole group, which is bounded by the open workload
Q5, Q6	Range scan of the completion partial index over ~12 weeks, then an aggregate

One dashboard load is therefore a handful of bounded scans plus one full count (Q1). Revisit if the table passes roughly a million rows (replace Q1 by maintained counts), or if the open workload stops being small relative to history.

Migration impact and recovery limits

Three migrations, in this order:

1. `AddProductionOrderCompletionTracking` — column, backfill, constraints (one transaction).

- **Migration type:** additive, then data-transforming (backfill) — the expand step of an expand/contract change (`ai/rules/database.md`); nothing is contracted later, because nothing old is replaced.
- **Steps:** `ALTER TABLE production_orders ADD COLUMN completed_at_utc timestamptz NULL` . A nullable column with no default is a catalog-only change, with no table rewrite. Then `UPDATE production_orders SET completed_at_utc = updated_at_utc WHERE status = 'Completed'` (DEC-004): for a completed order,

the last update is the completion save or a later edit of its notes or due date, so this is the best available evidence and never earlier than `created_at_utc`. Then both check constraints.

- **Locking:** the constraints are added validated, in the same transaction, which scans the table under the `ACCESS EXCLUSIVE` lock the `ALTER` already holds. At the sizes above, that scan takes well under a second. On a much larger table the constraints would be added `NOT VALID` and validated in a separate migration, which needs only a weaker lock.
- **Deploy order:** the backend version that sets `completed_at_utc` must be running **before** anyone can complete an order against the migrated schema. An old backend completing an order after migration 1 would violate `ck_..._matches_status`, and its save would fail with a server error rather than silently corrupt the figures. The existing Compose order already runs migrations with the backend stopped, then starts the new images (`deploy/README.md`), so no window exists there.
- **Side effect:** the backfill updates each completed row, which changes its `xmin`. A Screen A edit form already open on a completed order therefore gets the usual stale-save conflict (MSG-E009) on save and must reload. That is harmless and applies only to completed orders.
- **Data recovery limit:** reverting drops the column, so every completion time recorded after the migration is lost. It cannot be reconstructed except approximately, from `updated_at_utc`, which is how the backfill made it.
- **Rollback plan:** drop both constraints, then the column. The old backend then runs unchanged.

2. `AddProductionOrderDashboardIndexes` — the two partial indexes.

- **Migration type:** additive. `CREATE INDEX CONCURRENTLY IF NOT EXISTS` for each, via `migrationBuilder.Sql(..., suppressTransaction: true)`, exactly as 002_DB's index migration.
- **Non-atomic:** a failure part-way leaves an `INVALID` index that must be dropped (`DROP INDEX CONCURRENTLY`) before retrying. Check with `SELECT indexrelid::regclass FROM pg_index WHERE NOT indisvalid;`.
- **Data recovery limit:** none — no data is written.
- **Rollback plan:** `DROP INDEX CONCURRENTLY IF EXISTS` for both. Reverting only makes the dashboard slower, never wrong.

3. `SeedDashboardDemoHistory` — demo history (one transaction).

- **Migration type:** data-transforming and data-inserting, guarded (see Guards above). It re-dates up to 16 seeded rows, inserts up to 44, and advances the seed year's counter.
- **Data recovery limit:** the re-dated rows' original `created_at_utc`, `updated_at_utc` and `completed_at_utc` values — themselves demo values relative to an earlier run date — are overwritten and not restored on revert. Real orders are never touched.
- **Rollback plan:** `DELETE FROM production_orders WHERE id::text LIKE '0197e4a0-0000-7000-8002-%'` removes the 44 inserted rows. The counter is left advanced, so no order number is ever reused. The re-dated 16 keep their new timestamps.

- **Screen B impact:** the demo database grows from 80 to 124 orders. Screen B has no new behavior, but WI-003's documents and tests that state seeded figures — 002_DB's seed table, and tests asserting the 80-row total, the status spread or the 00081 next number — are updated in plan revision 2 to the new figures. Its tests that assert relative dates are unaffected, because no seeded active order's due date changes.

Open decisions

Decision	Options	Recommendation	Status
Completion tracking: column or history table	column; history table	column	decided — work-items/WI-004/decisions.md DEC-003
Backfill of existing completed orders	seed + backfill from <code>updated_at_utc</code> ; leave NULL	seed + backfill	decided — DEC-004
Demo history: add orders, re-date existing, or convert	add ~40; re-date 16; convert some; both add and re-date	both	decided — DEC-013 (user)
Active orders due beyond the 8-week look-ahead in the seed	add 4; none	add 4	decided — DEC-014 (Claude; open to user objection at 003_DB review)
How the dashboard gets one consistent snapshot	<code>REPEATABLE READ READ ONLY</code> transaction; one CTE statement; no guarantee	transaction	decided — DEC-015
Enforce the completion rules in the database	two CHECK constraints; application only	constraints	decided — recorded here (both rules hold regardless of the clock; see Constraints)
Index for the status counts	none; status index; maintained counters	none until ~1M rows	decided — recorded here